

REG NO: SCS3/147863/2024
NAME : KAMAU DAVID

WEKA ASSIGNMENT Q3

Using the diabetes dataset OR heart dataset (select either from the weka list of datasets).
Study the dataset first. Now set the Training/Testing data partition as 70%/30%.
We will compare 2 algorithms – kNN and J4.8 on classification

(a) **Obtain the accuracy and record the confusion matrix. Use the default algorithm settings.**

ANS:

The **Diabetes** dataset was used for this assignment

J48 Algorithm

Correctly Classified Instances – 76.5217%

Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
122 36 |  a = tested_negative
 18 54 |  b = tested_positive
```

IBk Algorithm

Correctly Classified Instances – 73.4783%

Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
126 32 |  a = tested_negative
 29 43 |  b = tested_positive
```

(b) **From the confusion matrix, which class appears to be more difficult to classify for each of the datasets?**

ANS:

Based on the confusion matrices above, the **tested_positive** (class b) is the more difficult class to classify for both algorithms.

This is because :

For **J4.8**, the error rate for predicting negative cases (class a) is roughly 22.8% (36 errors out of 158) while the error rate for predicting positive cases (class b) is higher at **25.0%** (18 errors out of 72).

For **kNN (IBk)**, the error rate for predicting negative cases is roughly 20.3% (32 errors out of 158). The error rate for predicting positive cases is significantly higher at **40.3%** (29 errors out of 72).

From the above information, we can safely say that both algorithms struggle more with positive cases, likely because there is less training data available for that specific class.

(c) **Change some parameters in each of the algorithms and note if the performance improves.**

ANS:

Summary:

J4.8: Changing the parameters did **not** improve performance. Increasing the minNumObj to 10 and 15 actually caused the accuracy to drop to 75.6522%. The best accuracy remains the default at **76.5217%**.

kNN (IBk): Changing the parameters **did** improve performance. Increasing the K-value to 5 raised the accuracy from the baseline of 73.4783% up to **74.7826%**.

Detailed explanations are here below:-

Tuning J48:

1) **Changing minNumObj parameter from 2 to 5**

The accuracy i.e. Correctly Classified Instances still remains at 76.5217%.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
124 34 | a = tested_negative
 20 52 | b = tested_positive
```

2) **Changing minNumObj parameter from 5 to 10**

The accuracy i.e. Correctly Classified Instances drops to 75.6522%.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
125 33 | a = tested_negative
 23 49 | b = tested_positive
```

3) **Changing minNumObj parameter from 10 to 15**

The accuracy i.e. Correctly Classified Instances drops to 75.6522 %.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
125 33 | a = tested_negative
 23 49 | b = tested_positive
```

Tuning Ibk:

1) Changing the Knn parameter from 1 to 3

The accuracy i.e. Correctly Classified Instances still remains at 73.4783 %.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
132 26 |  a = tested_negative
 35 37 |  b = tested_positive
```

2) Changing the Knn parameter from 3 to 5

The accuracy i.e. Correctly Classified Instances *rises* to 74.7826 %.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
136 22 |  a = tested_negative
 36 36 |  b = tested_positive
```

3) Changing the KNN parameter from 5 to 7

The accuracy i.e. Correctly Classified Instances drops to 73.0435 %.

Its Confusion Matrix :

```
=== Confusion Matrix ===
  a  b  <-- classified as
133 25 |  a = tested_negative
 37 35 |  b = tested_positive
```

(d) **Of these two algorithms, which one is the better classifier?**

ANS:

J4.8 is the better classifier for this specific dataset and split. Even after optimizing kNN to its best performing state ($K=5$), it still could not beat the baseline decision tree.

(e) **Using the three most relevant attributes as can be depicted from Decision tree algorithm, now create a KNN model and note if the performance improves. (You can do this by removing some attributes by checking them off when viewing the dataset)
Document your results clearly.**

ANS:

1. Identification of Relevant Attributes

*Based on the root and primary nodes of the J48 decision tree model generated in section (a), the three most highly predictive attributes for this dataset were identified as **plas**, **mass**, and **age**.*

2. Methodology

To conduct feature selection, the dataset was filtered to retain only these three critical attributes alongside the target class label. All other attributes were removed to eliminate potential dimensionality noise.

The IBk (kNN) algorithm was then re-evaluated on this reduced feature space using the established 70%/30% training and testing data partition.

3. Results and Observations

The impact of feature selection on the model's performance varied significantly depending on the hyperparameter K (number of nearest neighbors):

- ▶ **When $K = 1$ (Default):** The classification accuracy **decreased** from the baseline of 73.4783% down to 70.8696%.
- ▶ **When $K = 5$ (Tuned):** The classification accuracy **improved** from the previous optimum of 74.7826% up to **75.2174%**.

4. Conclusion

Feature selection successfully improved the overall performance of the kNN classifier, but only when the algorithm was appropriately parameterized. Removing less relevant attributes reduces

noise, which allows the kNN algorithm to calculate more meaningful distance metrics between data points.

However, in this compressed feature space, relying on a single neighbor ($K=1$) rendered the model overly sensitive to local outliers, causing a drop in accuracy. By utilizing a more robust neighborhood consensus ($K=5$) alongside the filtered data, the model achieved its highest kNN accuracy of 75.2174%.